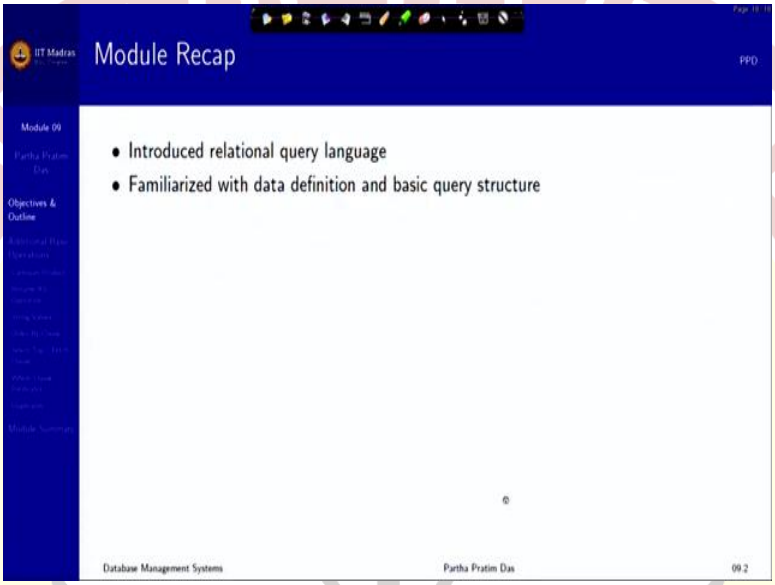
IIT Madras

ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Introduction to SQL/2

Welcome to module 9 of Database Management Systems course in the IIT Madras Online B.Sc. Program.

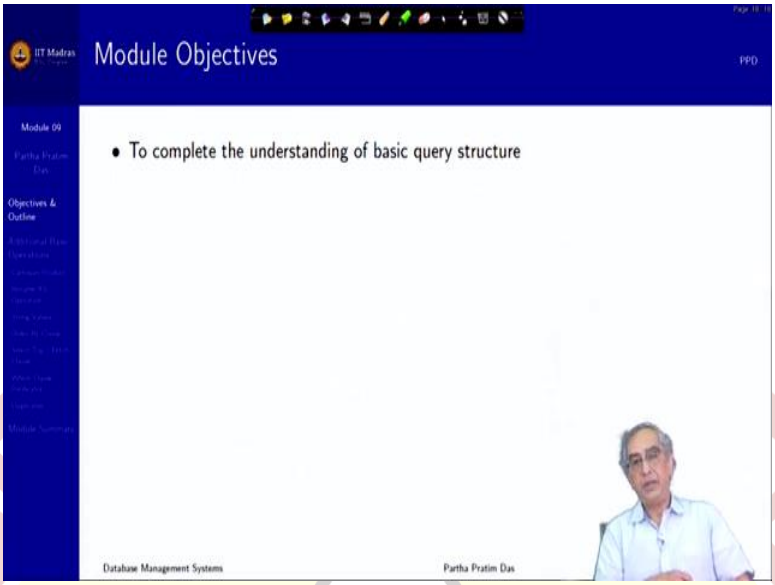
(Refer Slide Time: 0:31)

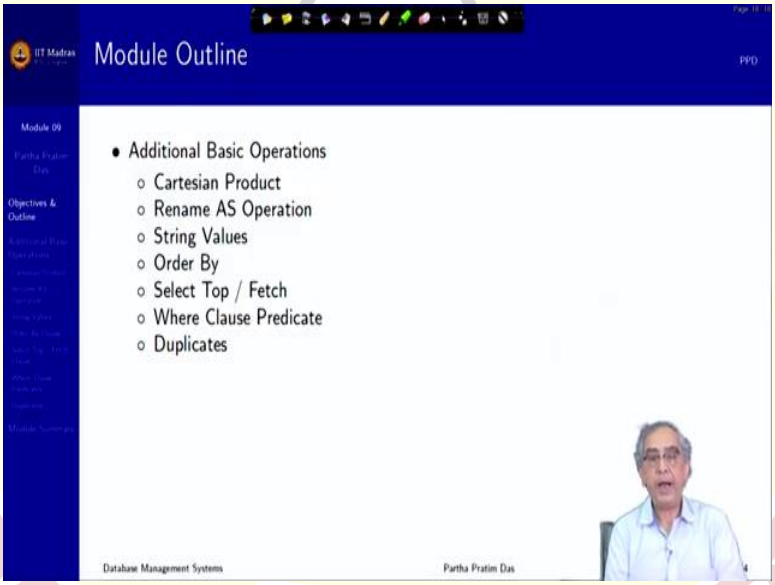


We in the last module had started going about discussing the relational query language, the actual commercial language known as SQL and we introduced the basic structure of how to create tables and do basic manipulation on the table Schema and we have discussed the basic structure of 'select from where' query.

I know you may not yet be comfortable with this. Wait, after we have introduced certain part of SQL, certainly we will have a more detailed walk through with examples for each one of this so that you can get more and more comfortable with it.

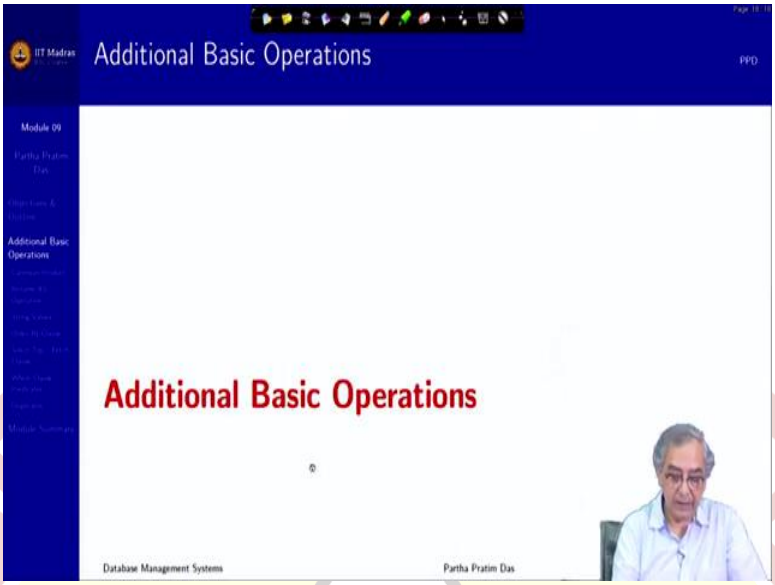
(Refer Slide Time: 1:15)

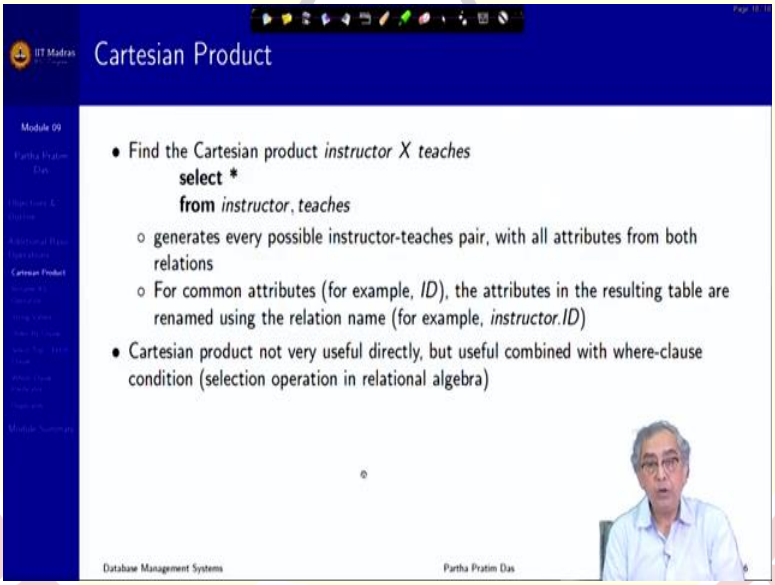




Continuing on that we will talk about several other features of the basic query structure which can make, which enables you to write varied different forms of query and in some cases makes it easier for you to write the query. So, under this we have several things that we will discuss continuing on from the Cartesian product that we had talked about towards the end of last module and you will get the list on the left.

(Refer Slide Time: 1:51)



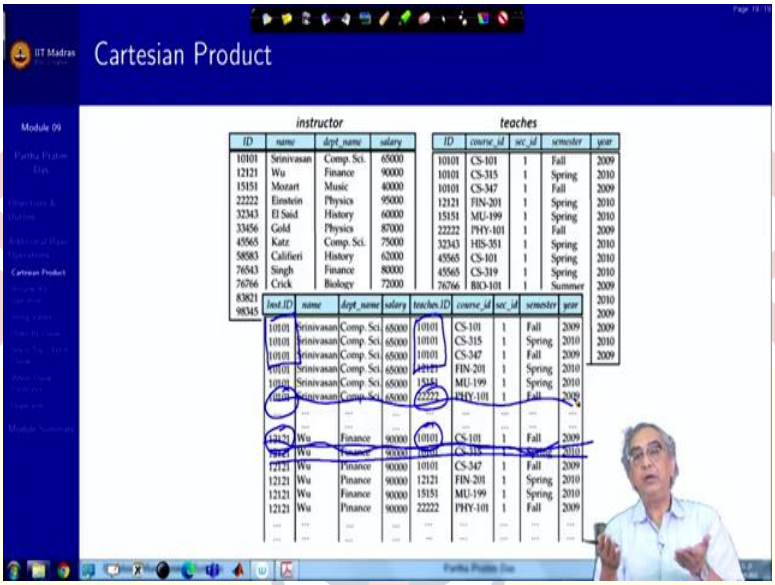


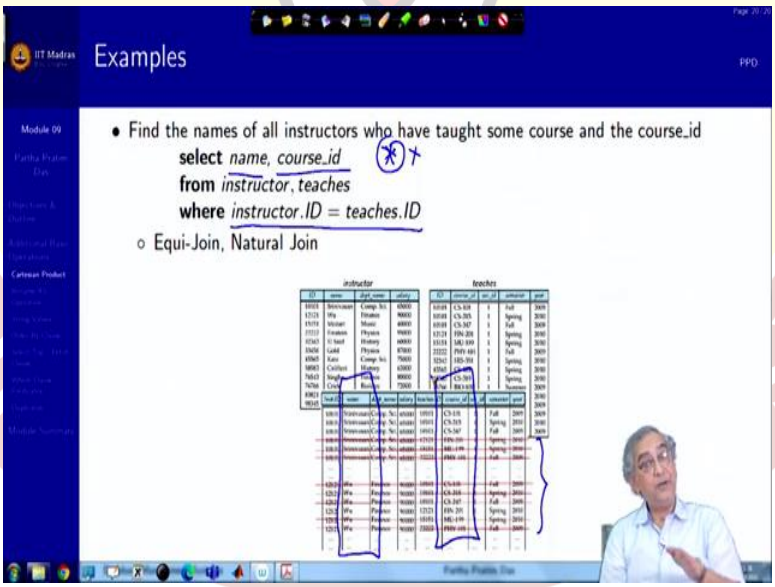
So, let us get started. So, this is what we are calling Additional Basic Operations. So, we have observed that the Cartesian product, like if I have two relations; instructor and teaches, the Cartesian product is a combination of all tuples that instructor has and all tuple that the teacher has.

So, if I take, if I specify in the 'from clause' that I have instructor and teaches and specify a wildcard in the select, which means take all columns then what it will simply mean that a big table will be formed having the columns of instructor, having the columns of teaches and having all possible combinations of records.

Naturally, if there is any column whose name is identical between the two relations, then it will be qualified by the name of the relation. So, for example, ID, the ID of the instructor will be common, so what is ID in instructor and in teaches will be made instructor.ID and teaches.ID respectively. So, let us see what will happen with this.

(Refer Slide Time: 3:14)





So, this is what we saw this earlier also. And as we mentioned that this in particular is not very useful. So, what we can, have to do is why, where is the information actually? The information about the instructor is on the instructor table. The information about the instructor for courses are on the teaches table where the instructor is modeled simply by his or her ID.

And now what you are trying to do? You are trying to relate this to information so that, for example, I can get the list of courses that a particular instructor is teaching, which is neither available in instructor nor available in teaches, because in teaches you have the courses against the ID, in the instructor you have the name of the instructor against the ID.

So, if I tell you what are the courses professor Srinivasan is teaching you cannot answer that either from instructor or from teaches, but you can answer them if you take the combination; but the combination produces naturally. It is just a blind combination, so it produces lot of meaningless tuples; like these are tuples where the ID of the instructor and the ID of the teaches do not match, idea of the instructor and ID of the teaches do not match.

But these are useful because there the ID of the instructor and the ID of the teaches match. So, what do you want to do if the instructor ID between the instructor table and the teaches match that means that this particular instructor is teaching this particular course and that is what we want. So, what we want to do? We want to eliminate all these unwanted records. So, how do you specify that this is a simple Boolean condition?

That instructor.ID is equal to teaches ID, so which we can comfortably write as a part of the 'where clause', we write instructor.ID is teaches.ID and all these rows you can see which are struck off by red are actually struck off. What is left is everywhere, you have just the records where the instructor.ID and in teaches.ID are same.

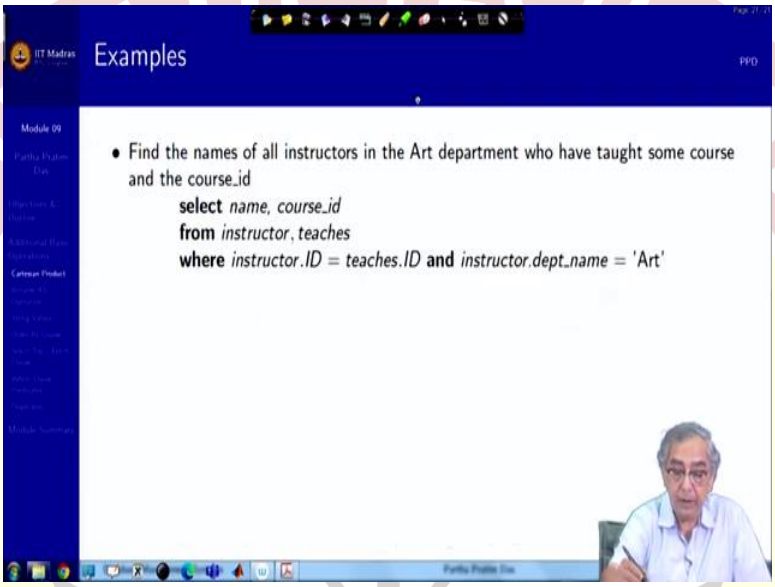
And then we are not actually interested in the employee ID of the instructor. We are interested in the name. So, instead of earlier we were doing star we do not need that, but you say the name and the course ID. So, it tells us, so it just takes the name and the course ID, these two columns. So, it will tell me Srinivasan CS101, Srinivasan CS315, so tells me what course is professor Srinivasan is teaching.

We can take, we do not have in this the names of the courses. So, which means if I want the name I will again have to do a similar operation with a table which has the name of the course against the course ID, which is precisely the courses table and we will have to do another similar operation. This operation is known as Natural Join.

We talked about in the relational algebra case and we will see lot more depth of this Natural Join because it is useful in in combining information from multiple tables. Just for your, just to sensitize you this example is showing the join of two tables, you can join three tables, four

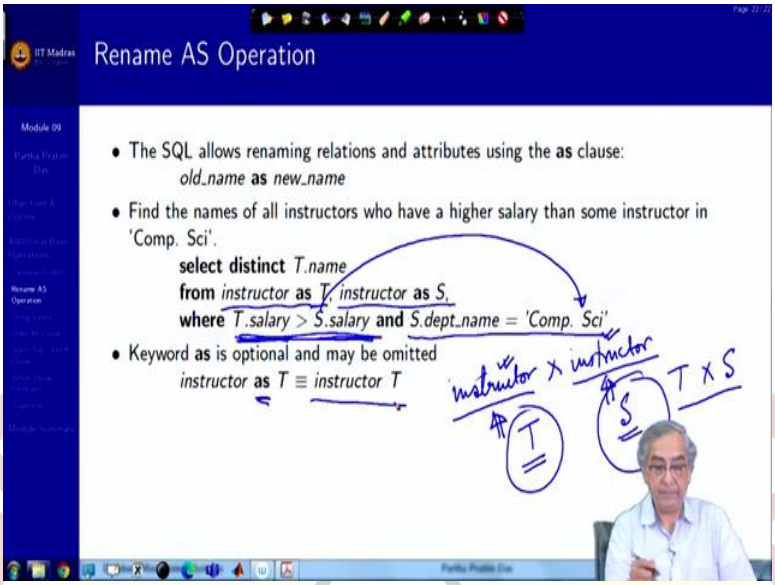
tables any number of tables but we will see over time that what are the consequences of joining multiple tables because more and more tables you join the first Cartesian product generates a huge number of records and then you eliminate most of them, so you have to decide what is the order in which you should do the join and so on so forth, we will talk about those but I am just sensitizing you about the this question in mind that you should bear always.

(Refer Slide Time: 7:58)



Now, this is a refinement of the earlier query. So, it says that ‘find out the courses being taught by all instructors in the art department’. So, the first part of the where clause condition is doing the basic natural join and the second part in that process is specify that the instructor has to come from the department of art, which was already available in the instructor table.

(Refer Slide Time: 8:36)



Now, often times you need to, when you do the select you would like to give a different name either to the attribute or you may need to give a name to the relation itself, so this is where we use the ‘as’. Now, if you look at this particular example; what we are trying to do is we are trying to do, I am sorry, we are trying to do instructor cross instructor.

Now, as we do instructor cross instructor two kind of things will happen; one is in the output projection on the select list how will the ID, how will the attributes be identified? For example, the ID of this instructor table is instructor.ID; this also is instructor.ID, earlier we were doing instructor and teaches, so it was easier, the relation names were different, the table names were different.

So, this is one problem. The second problem is if I want to put some condition between the attributes of this instance of the instructor table and the same instance of the instructor table; that is if I want to do the operation within a single table I will not be able to specify because which one am I talking of, the left one or the right one.

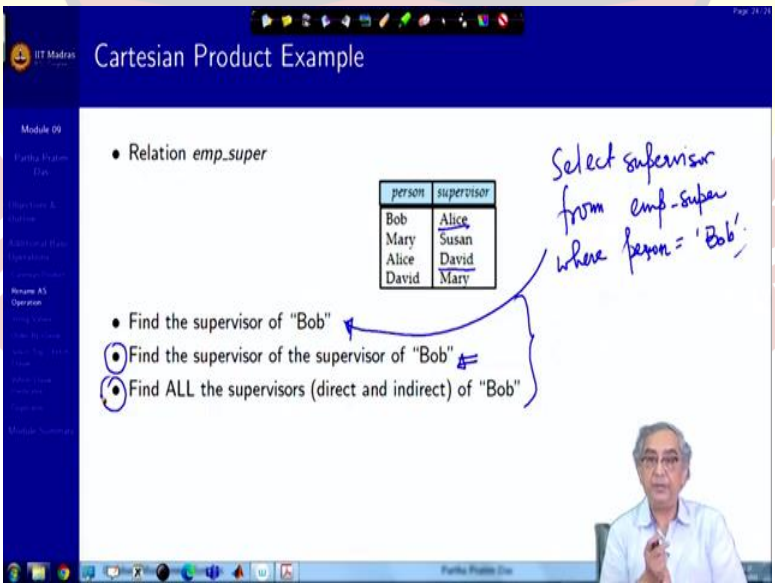
So, what we can do is something very simple; we just give them two different names. So, we say this is T, this is S. So, this actually is T cross S. So, now when you talk about T.salary, you are talking about this instance, when you are talking about S.salary you are talking about this instance. So, now what you are saying it is a join, it is a Cartesian product of T and S, all possible.

So, you are taking a table and creating a Cartesian product with itself, joining the table with itself. Now, as we do that then we say that T.salary is greater than S.salary. So, what you are saying that any record which is coming from T, I will keep that provided the salary value is greater than the salary value here. So, what will happen? This will retain the highest salary. Now, I want to do it specifically with reference to certain faculty.

So, I say further that S.department name is computer science. So, which means that in this S we have done a selection, which is just computer science faculty; in T we have not done any selection, so there is computer science faculty as well as other department faculty; so for all faculty I will select those whose salary is higher than some faculty, some instructor in the computer science department, not all because this condition can get satisfied for any.

But when it gets satisfied this S refers to that the department name has to be computer science, so he has to be a, he or she has to be a computer science faculty. So, all those faculty who have their salary higher than some faculty of computer science will be selected. So, this is you can do it and you can see that using the as keyword is optional, you can just write instructor T and that will meet instructor as T. It is kind of a renaming as you can see, but that is very important to do self-operation.

(Refer Slide Time: 12:58)



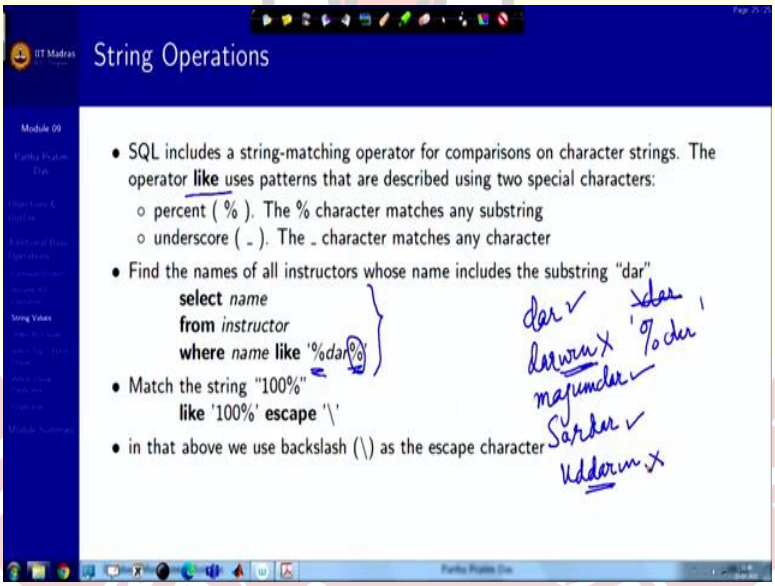
So, this is, this I will leave for you. So, this is a table which gives you the person and the person supervisor. So, let us say if I have Bob, thus Bob's supervisor is Alice. It is not difficult to find that because you can always do select from, select supervisor, so what will you do you? You will write select, I am just showing you this one.

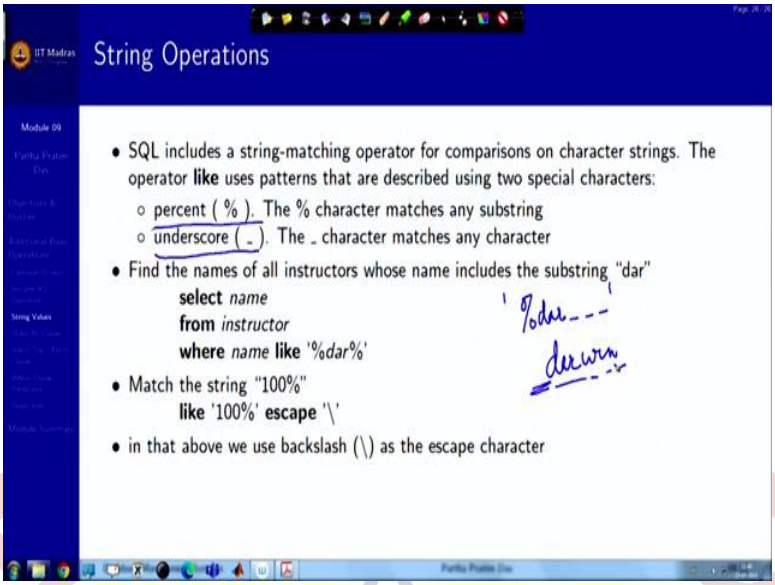
What do I want to find out? Supervisor, so select getting cluttered. Select supervisor from the name of the table emp-super, where what is the condition, you want to find it for Bob, so where person equal to Bob; this will give you this. The question is how do you find the supervisor of the supervisor of Bob?

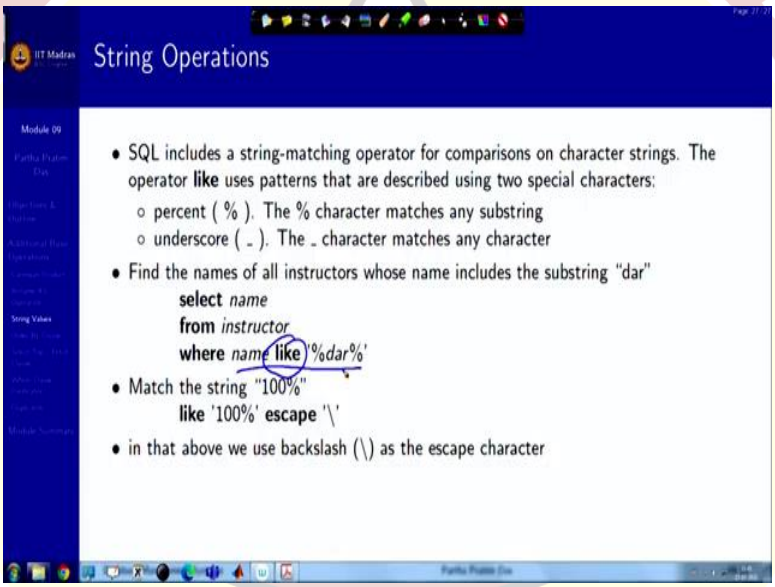
So, supervisor of Bob is Alice and the answer to this question is actually meaning that who is the supervisor of Alice which is David, I can go even further, I can say so supervisors, all supervisors of Bob in the entire hierarchy. So, think about how can you use Cartesian product to find an answer to this and this.

I am not telling you the answer right now, we will discuss it at some later point, but I have just shown you the simple way. But Cartesian product will help you because you can form combinations and see as to what should be the condition so that you can get the supervisor of supervisor of Bob and how you can get all supervisors of Bob.

(Refer Slide Time: 15:16)







The next thing I want to tell you about is strings. Strings are very important because lot of data are of string type, names, even IDs, department name, course name, all these. So, we often have the exact string to match like computer science we used, then it is easy, put it in quotes, use but often we want to match partially or we just know a part of the string and we want to find all of that.

So, for that we have what is known as the like operator. So, I will tell you afterwards what are these; just look at this example. So, we are saying that select name, find the instructors whose name is like "dar", Majumdar, will get selected. Now you do not know, you want to say that "dar" could occur as in Darwin at the beginning, "dar" could occur as Majumdar at the end.

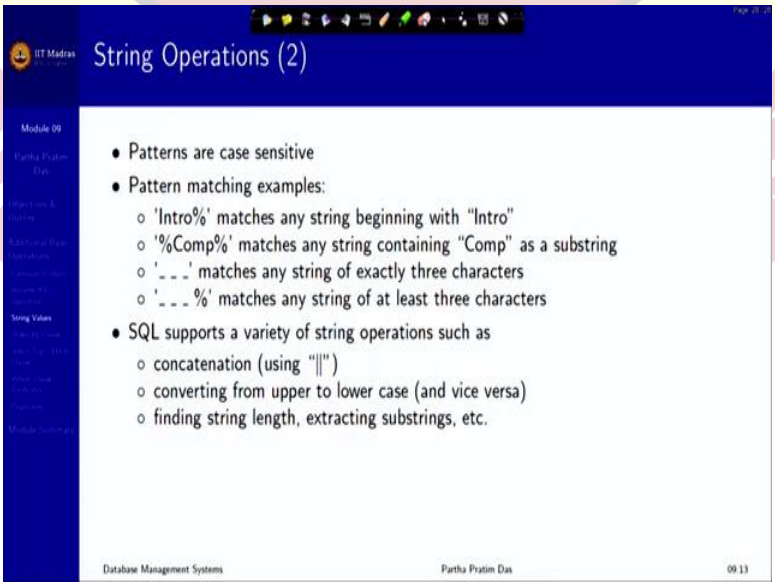
So, you want to say that “dar” could be anywhere within the name, so you write percentage. A percentage means that it will match any string including the null string. So, “dar” could be preceded by anything. Similarly, I put a “dar” percentage here to say that it could be followed by anything as well. So, it will match dar, it will match Darwin, it will match Majumdar, it will match Sardar, and so on.

In fact, it could be in the middle. It will match, there in where “dar” is here. If I just write instead of putting percentage, I am sorry, forget this. Instead of putting percentage on both sides if I just put it in the front it will match this, but it will not match this because it needs a trailing match. It will match this, it will match this, but it will not match this.

So, that is how it works. Now, I can have, specify any number of characters or I can specify these many characters I want, so I can say that I want percentage “dar”, so this will match Darwin but will not match the others that we are talking about because it says that there could be something in front, whatever, then “dar” and there will have to be three characters after that. So, in this way there are several features you can extend strings matching.

But like is the basic operation through which you can write conditions and the result of always doing this are picking up those tuples where the match has happened, those are the true ones. I am sorry.

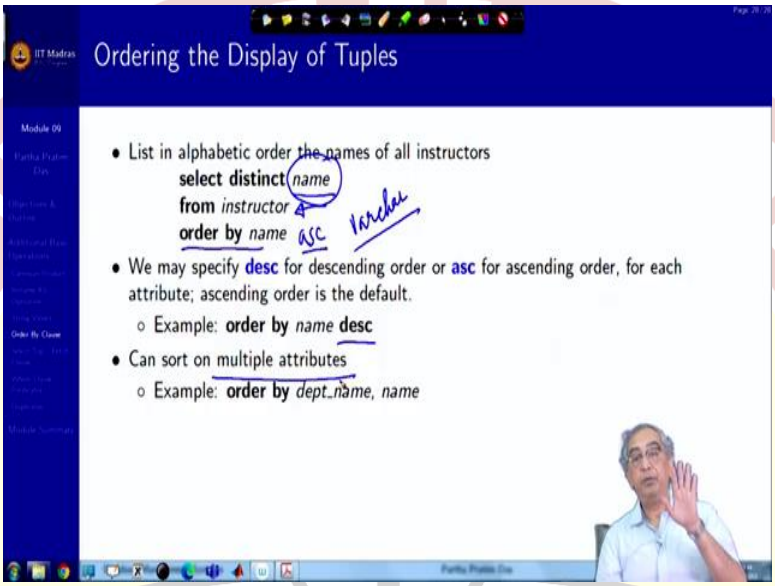
(Refer Slide Time: 19:14)



So, these are different examples of what I talked of where the percentage is only at the end or on both sides. There are fixed number of characters involved and please remember that when

it comes to strings or patterns everything is case sensitive unlike general SQL keywords and names. But anything that you write within quotes the string this is case sensitive, which means uppercase and lowercase will be treated differently. And in addition SQL supports different other supporting operations like concatenation, convert lower case, convert to upper case, like that.

(Refer Slide Time: 20:00)



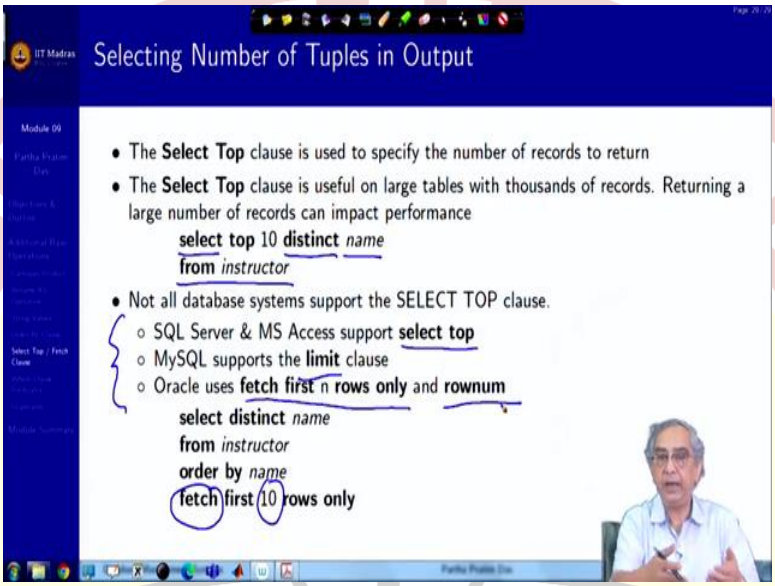
The next is suppose you have generated the output and often what we want is in an output if it comes in an arbitrary order it becomes difficult to read that. So, many times the lists are provided in terms of say as it is called lexicographical order that is in the order in which things happen in the dictionary or some other specific order, maybe in terms of rank, this is the topper of the class CGPA, so it is ordered by CGP.

So, we can do the same thing in SQL by using a clause known as order by, so what you are saying, you take, you are taking from instructor, selecting names, you want the names to be distinct, so two instructor with the same name will appear as one entry only and then you want them in the order of the name from the name you know.

What is name? Name is type is varchar, so it is basically a string, so it will be ordered according to the string logic. Now, obviously there is a question of whether I should start from A and go to Z or start from Z and go to A, increasing decreasing, smaller to larger, larger to smaller, so by default it is ascending.

But if you want descending specifically, you can write or even ascending you can write specifically, it has no special effect on the output but maybe makes it easier to read, but one thing has to be kept in mind that you can order the table by one attribute in addition; you can order them by combination of attributes also, that is you can order by a tuple as well. So, you can sort on multiple attributes as well.

(Refer Slide Time: 22:18)



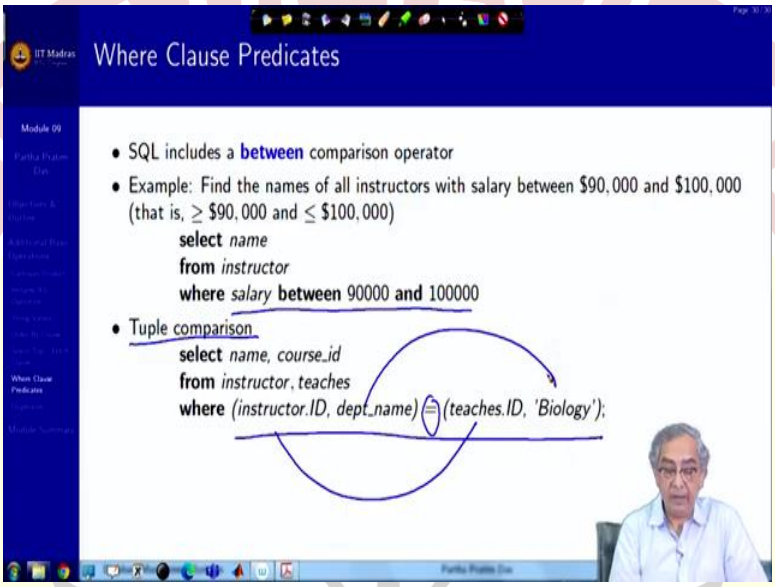
These are, I mean, these are kind of different ways to make your output table more easy to understand, easy to read and convenient to use. So, here is another is which says select top, name tells you what it means. It says that the basic query is from instructor, select name from instructor, then you want distinct names and then what you are saying is select top 10.

Top 10 means the first 10. Because often you will see that if the list is long you cannot present it in one go. For example, in your inbox, in Gmail, the mail some 20, 30, you may have put a choice number there but it is limited, maybe 20, 30, 50 something. In your account transaction the records will be shown say 20 records or 40 records like that.

So, this is to get a part of the output table out. So, you can do this by several different ways in different databases, select top 10, top is one way. Another which is actually very dominant in SQL is doing this by fetch. I mean, this is not a different thing but as you can see, I told you that not everywhere all database systems follow the same kind of language forms, language keywords, so SQL server, MS Access they use select top syntax.

MySQL use what is called limit and Oracle, which more closely follows the SQL standard says fetch, says ‘fetch first n rows only’, so ‘fetch first 10 rows only. In this way you can select any specific number of rows, you can specify it by row number also and so on. So, it basically says that you have a big list and you can choose the part that you want in the output and then you can again do the query take out another part. So, it is kind of next, next as you go in pages, the similar thing can be very easily written here.

(Refer Slide Time: 24:59)

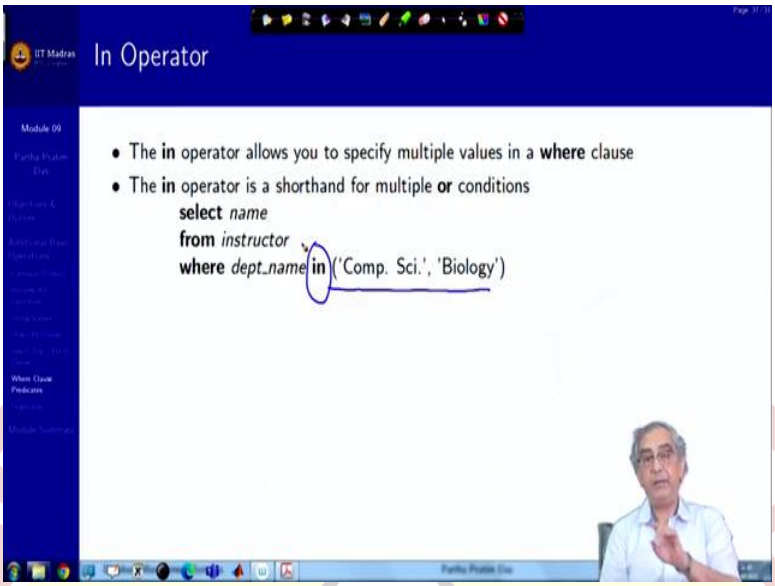


I will talk about some more convenient stuff in the where clause, for example, many a times you need to put a range as a condition. So, you are saying the salary is between 90,000 dollars and 100,000 dollars. Obviously you can write it as two parts of the query; that salary greater than equal to 90,000 dollar and salary less than equal to 100,000 dollar but you can make it easier to write and as well as to read by saying that salary between this and this.

So, between gives you a range, it has a lower value and a higher value and within that, if the value falls within that it becomes true, if it is outside of that it becomes false. The other convenience is in where clause is tuple comparison where you can write comparison if, of course, the comparison is the same like this, which means that instructor ID is equal to teacher ID and department name equal to this.

Certainly instructor ID equal to teaches ID and department name equal to biology this is more compact way of saying that this pair has to be equal with this pair. Typically used for equality, can be used for others, but is not very meaningful.

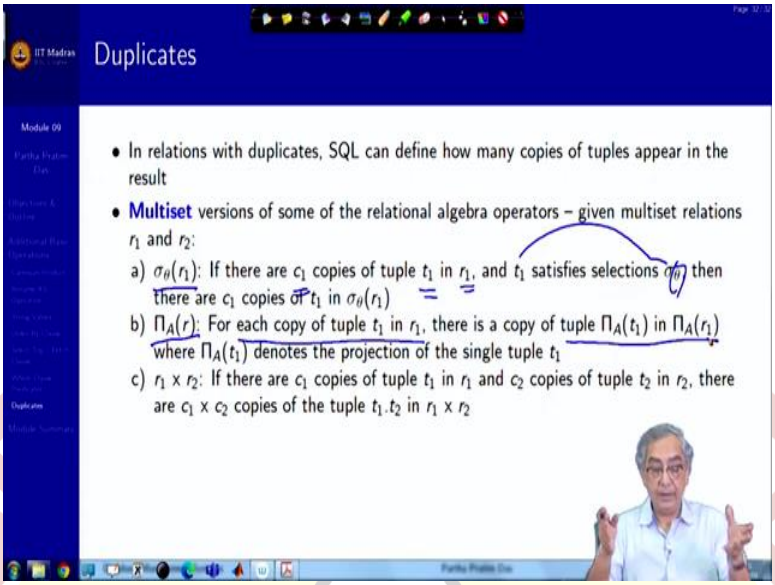
(Refer Slide Time: 26:32)



There is another way of specifying, simplifying the predicate in the where clause, which is known as in, in is as in set membership. So, if you have multiple values that you want to specify; that you want to say that department name is computer science or the department name is biology, so you could have 10 such or the department name is say mechanical, then get the instructor.

Instead of writing so many clauses and or you can simply write put them in a list and put in, so which means this will be true if department name is computer science or if department name is biology. So, these are, basically these are not adding new features, but this just makes your coding, programming in SQL easier.

(Refer Slide Time: 27:44)



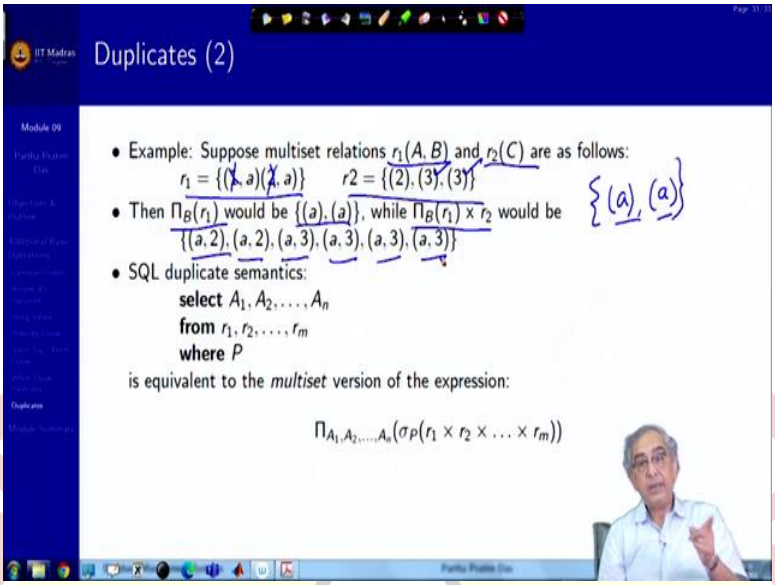
Now, we come to the question of duplicate. We have made contradictory statements. We said that in relational algebra duplicates are not allowed, in SQL we said duplicates can be there, so just, I mean, in relational algebra if we look at it is dependent on set theory and set does not allow duplicates, so there is an extension to that which is called multiset.

It is a version of some relational algebra operation which can be work as multiset. In multiset what it means is you can have multiple elements which are identical and therefore you can have multiset operations. So, for example, if you have say a select and say there is a tuple t1 which occurs in c1 number of copies in the relation r1 and t1 satisfies the condition theta.

So, it will get selected. So, in a multiset situation it is not one record which will get selected, but all of these records, all c1 records will get selected. Simple thing, is just that the distinctness removal is not considered, in every computation you do not consider that it is a set and everything is distinct, it could be multiple.

So, similarly if you do projection and what happens when you do projection? There may be, on that column many tuples may have the same value, so each copy of tuple t1 in r1 there is a copy of projection on this t1 on r1. So, if there are multiple ones those multiples will be retained, similar thing happens for Cartesian product.

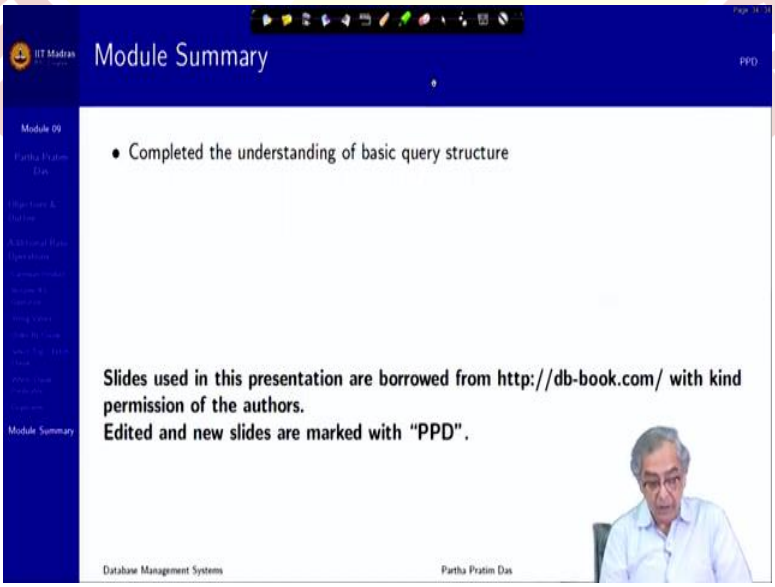
(Refer Slide Time: 29:50)



So, this is kind of a little bit of extension in terms of the duplicate management with multiset, so this is just an example here I have tried to give. So, there are two relations r1 and r2, r1 has two fields, r2 has, but there are duplicates here and then I take a projection. As I take a projection these will go, so what I will get is a set having two a's.

This is a multiset. And now if I take a cross product with r2 then I will have two a's on the left side. 1, 2 and two 3's on the right side, so you have a2 a2 from two a's, you have a3 a3 a3 a3 from two a's across two 3's. So, that is basically the notion of multiset and that is what SQL internally follows.

(Refer Slide Time: 30:52)



So, with this we come to the end of this module. Thank you for your attention and I would just like to tell you that you may be feeling little loaded with this language, so much syntax and all that going on, just have some patience and we will soon, in the next week will soon come, help you with more supporting examples and tutorial.

But we cannot do that until you have learnt the basics of the language, always keep these slides handy with you and you will be able to refer to them as needed. Thank you very much and see you in the next module.

INDIAN INSTITUTE OF TECHNOLOGY MADRAS
सिद्धिर्भवति कर्मजा